

L5,6. Aplicații ale rețelelor neuronale MLP

În acest capitol sunt descrise câteva aplicații tipice ale rețelelor neuronale MLP.

Recunoașterea orientării unei piese

În continuare vom descrie o problemă practică de recunoaștere de forme, care se întâlnește în robotică. Pentru soluționarea acestei probleme vom folosi o rețea neuronală MLP cu un singur strat ascuns.

Pentru ca un robot să poată apuca cu degetele sale o piesă, trebuie să-i cunoască orientarea. În mod normal, piesele îi vin robotului cu orientări aleatoare (de la un vibrator). O cameră de luat vederi captează o imagine a fiecărei piese primite de la vibrator de către robot. Sistemul trebuie să proceseze datele achiziționate și să atribuie piesei o anumită orientare dintr-un număr total de orientări posibile. Spre exemplu, dacă toleranța admisă în orientarea degetelor robotului este de $-15 \dots +15$ grade, față de direcția ideală, sunt în total $360 / 30 = 12$ clase de orientări posibile. În această problemă, vom considera imaginea preluată cu o cameră de luat vederi alb - negru, iar piesa (un șurub) este centrată. Considerăm 4 clase de orientări

Bitmapul imaginii achiziționate, ideal, pentru una din cele 4 orientări posibile, pentru o imagine de 20x20 pixeli, este următorul: (1=pixel negru, 0=pixel alb)

[illegible]

O imagine achiziționată, poate avea însă zgomot suprapus (pixeli inversați ca valoare). Rețeaua neuronală trebuie să atribuie corect orientarea, chiar când imaginea achiziționată are un nivel rezonabil de zgomot.

În general, înainte de a fixa pe intrările rețelei neuronale o imagine achiziționată (tipic, pentru fiecare pixel, un neuron de intrare), imaginea trebuie preprocesată. O posibilitate de prelucrare este și reducerea dimensiunilor imaginii.

Ca exemplu, vom preprocesa elementar o imagine de 20x20 pixeli, astfel încât să ocupe 5x5 pixeli. Considerăm imaginea de 20x20 pixeli stocată în fișierul text *mare.txt*. Punctele albe din imagine sunt reprezentate prin caracterul '0', cele negre prin caracterul '1'. Caracterele sunt separate prin spații. Imaginea redusă la dimensiunile 5x5 pixeli, o vom salva în fișierul *mic.txt*.

Algoritmul folosit pentru preprocesare este următorul: o matrice de 4x4 pixeli din imaginea inițială (20x20) se reduce la un pixel în imaginea preprocesată. Valoarea pixelului este 1 dacă cel puțin 8 puncte (jumătate) din cele 16 ale fragmentului 4x4, au valoarea 1.

În continuare vom prezenta un program ce simulează o rețea neuronală MLP binară, folosită pentru a rezolva o problemă asemănătoare cu problema de robotică descrisă anterior. În acest caz imaginea are 5x5 pixeli. Se va folosi o rețea neuronală ce are 25 de neuroni de intrare, 20 de neuroni în stratul ascuns și 4 neuroni de ieșire. Cele 4 ieșiri (Y0,Y1,Y2,Y3) sunt asignate fiecare, uneia din cele 4 orientări distincte pentru șurub. Vom folosi 12 șabloane de

antrenare (câte trei pentru fiecare poziție distinctă a piesei). Aceste șabloane se inițializează direct în variabile de memorie.

```
import java.util.*;
import java.io.*;
class Robot5x5
{
    static final int DIMENSIUNE=5;
    //dimensiunea matricii patrate de pixeli, aplicate pe intrari
    static final int NR_NEURONI_IN=DIMENSIUNE*DIMENSIUNE;
    static final int NR_NEURONI_OUT=4;
    static final int NR_NEURONI_HIDDEN=20;
    static final int NR_TOTAL_SABLOANE=4*3;
    //cate 3 sabloane, pentru fiecare orientare.
    static final double EROARE_IMPUSA=0.01;
    static final int NR_MAXIM_EPOCI=40000;
    static double w1[][]=
        new double[NR_NEURONI_HIDDEN][NR_NEURONI_IN];
    static double w2[][]=
        new double[NR_NEURONI_OUT][NR_NEURONI_HIDDEN];
    //sabloaneIn – contine intrarile a 12 sabloane fiecare avand cate
    //25 de valori binare. Este deci un array ce contine 12 array-uri:
    static int sabloaneIn[][]=
        {
            {1,1,1,1,1,
             0,0,1,0,0,
             0,0,1,0,0,
```

0,0,1,0,0,
0,0,1,0,0},

{0,1,1,1,0,
0,0,1,0,0,
0,0,1,0,0,
0,0,1,0,0,
0,0,1,0,0},

{0,0,0,0,0,
1,1,1,1,1,
0,0,1,0,0,
0,0,1,0,0,
0,0,1,0,0},

{0,0,0,0,1,
0,0,0,0,1,
1,1,1,1,1,
0,0,0,0,1,
0,0,0,0,1},

{0,0,0,0,0,
0,0,0,0,1,
1,1,1,1,1,
0,0,0,0,1,
0,0,0,0,0},

$\{0,0,0,1,0,$
 $0,0,0,1,0,$
 $1,1,1,1,0,$
 $0,0,0,1,0,$
 $0,0,0,1,0\},$

$\{0,0,1,0,0,$
 $0,0,1,0,0,$
 $0,0,1,0,0,$
 $0,0,1,0,0,$
 $1,1,1,1,1\},$

$\{0,0,1,0,0,$
 $0,0,1,0,0,$
 $0,0,1,0,0,$
 $0,0,1,0,0,$
 $0,1,1,1,0\},$

$\{0,0,1,0,0,$
 $0,0,1,0,0,$
 $0,0,1,0,0,$
 $1,1,1,1,1,$
 $0,0,0,0,0\},$

```
{1,0,0,0,0,  
1,0,0,0,0,  
1,1,1,1,1,  
1,0,0,0,0,  
1,0,0,0,0},
```

```
{0,0,0,0,0,  
1,0,0,0,0,  
1,1,1,1,1,  
1,0,0,0,0,  
0,0,0,0,0},
```

```
{0,1,0,0,0,  
0,1,0,0,0,  
0,1,1,1,1,  
0,1,0,0,0,  
0,1,0,0,0}
```

```
};
```

//sabloaneOut – contine iesirile a 12 sabloane, fiecare avand cate

//4 valori (corespunzatoare celor 4 orientari):

```
static int sabloaneOut[][]=
```

```
{
```

```
{1,0,0,0},  
{1,0,0,0},  
{1,0,0,0},
```

```
        {0,1,0,0},
        {0,1,0,0},
        {0,1,0,0},

        {0,0,1,0},
        {0,0,1,0},
        {0,0,1,0},

        {0,0,0,1},
        {0,0,0,1},
        {0,0,0,1}

};

public static void main(String args[])
{
    boolean converge;
    //antrenare retea:
    initPonderi();
    converge=antrenare();
    if(converge==false)System.out.println(
        "Nu a convergat in nr. de epoci specificat !");
    else{
        System.out.println("A convergat. !");
        testare();
    }
}
} //main
```

```
private static void initPonderi()
{
    int i,j;
    Random r=new Random();
    //initializare primul strat de ponderi:
    for(i=0;i<NR_NEURONI_HIDDEN;i++)
        for(j=0;j<NR_NEURONI_IN;j++)
            w1[i][j]=r.nextDouble()-0.5;
    //initializare stratul al doilea de ponderi:
    for(i=0;i<NR_NEURONI_OUT;i++)
        for(j=0;j<NR_NEURONI_HIDDEN;j++)
            w2[i][j]=r.nextDouble()-0.5;
}

private static boolean antrenare()
{
    double yHidden[]=new double[NR_NEURONI_HIDDEN];
    double yOut[]=new double[NR_NEURONI_OUT];
    double eroareIesireSablonCrt[]=
        new double[NR_TOTAL_SABLOANE];
    double eroareTotala;
    int nrEpoci=0;
    boolean converge=false;
    boolean stop=false;
    int i,ex;
    while(!stop){
        for(ex=0;ex<NR_TOTAL_SABLOANE;ex++){
```

```

    calculIesiriNeuroni(yHidden,yOut,ex);
    eroareIesireSablonCrt[ex]=0;
    for(i=0;i<NR_NEURONI_OUT;i++)
        eroareIesireSablonCrt[ex]+=0.5*(yOut[i]-
            sabloaneOut[ex][i])*(yOut[i]-sabloaneOut[ex][i]);
    modificaPonderi(yOut,yHidden,ex);
} //for ex
nrEpoci++;
eroareTotala=0;
for(ex=0;ex<NR_TOTAL_SABLOANE;ex++)
    eroareTotala+=eroareIesireSablonCrt[ex];
System.out.println("Epoca nr: "+nrEpoci+
    " eroare totala= "+eroareTotala);
if(eroareTotala<=EROARE_IMPUSA)
    {converge=true;stop=true;}
if(nrEpoci > NR_MAXIM_EPOCI)stop=true;
} //while
return converge;
}

private static void calculIesiriNeuroni(double yHidden[],
    double yOut[],int ex)
{
    int i,j;
    double xTotal_i;
    for(i=0;i<NR_NEURONI_HIDDEN;i++){
        xTotal_i=0;

```

```
for(j=0;j<NR_NEURONI_IN;j++){
    xTotal_i=xTotal_i+sabloaneIn[ex][j]*w1[i][j];
    yHidden[i]=1/(1+Math.exp(-xTotal_i));
}

for(i=0;i<NR_NEURONI_OUT;i++){
    xTotal_i=0;
    for(j=0;j<NR_NEURONI_HIDDEN;j++){
        xTotal_i=xTotal_i+yHidden[j]*w2[i][j];
        yOut[i]=1/(1+Math.exp(-xTotal_i));
    }
}

private static void modificaPonderi(double yOut[],
                                   double yHidden[],int ex)
{
    double deltaOut[]=new double[NR_NEURONI_OUT];
    double deltaHidden[]=new double[NR_NEURONI_HIDDEN];
    int i,j,k;
    double sumaPonderataDeltaFii;
    double alfa=1.0;
    double eta=1.0;
    for(i=0;i<NR_NEURONI_OUT;i++){
        deltaOut[i]=(yOut[i]-sabloaneOut[ex][i])*
                    (1-yOut[i])*yOut[i];
    }
    for(i=0;i<NR_NEURONI_OUT;i++){
        for(j=0;j<NR_NEURONI_HIDDEN;j++){
```

```

        w2[i][j]=alfa*w2[i][j]-eta*deltaOut[i]*yHidden[j];

for(i=0;i<NR_NEURONI_HIDDEN;i++){
    sumaPonderataDeltaFii=0;
    for(j=0;j<NR_NEURONI_OUT;j++)
        sumaPonderataDeltaFii+=w2[j][i]*deltaOut[j];
    deltaHidden[i]=sumaPonderataDeltaFii*
        (1-yHidden[i])*yHidden[i];
}

for(i=0;i<NR_NEURONI_HIDDEN;i++)
    for(j=0;j<NR_NEURONI_IN;j++)
w1[i][j]=alfa*w1[i][j]-
        eta*deltaHidden[i]*sabioaneIn[ex][j];
} //modificaPonderi

private static void testare()
{
    int i,j;
    //iesiri neuroni din strat hidden:
    double yHidden[]=new double[NR_NEURONI_HIDDEN];
    //iesiri neuroni din stratul aut (cele calculate):
    double yOut[]=new double[NR_NEURONI_OUT];
    //intrarile pt. testarea retelei:
    int inTest[]=new int[NR_NEURONI_IN];
    double xTotal_i;
    Scanner sc=new Scanner(System.in) ;
    System.out.println("Dati cele " + NR_NEURONI_IN+

```

```
        " valori binare de pe intrari: ");
    for(i=0;i<NR_NEURONI_IN;i++){
        if(i%DIMENSIUNE==0)System.out.print("\n\n");
        System.out.print("Valoarea pixelului "+i+" : ");
        inTest[i]=sc.nextInt();
    }//for
    //Propagare inainte:
    for(i=0;i<NR_NEURONI_HIDDEN;i++){
        xTotal_i=0;
        for(j=0;j<NR_NEURONI_IN;j++)
            xTotal_i=xTotal_i+inTest[j]*w1[i][j];
        yHidden[i]=1/(1+Math.exp(-xTotal_i));
    }//for i

    for(i=0;i<NR_NEURONI_OUT;i++){
        xTotal_i=0;
        for(j=0;j<NR_NEURONI_HIDDEN;j++)
            xTotal_i=xTotal_i+yHidden[j]*w2[i][j];
        yOut[i]=1/(1+Math.exp(-xTotal_i));
    }//for i

    System.out.println("Orientare sus    : "+yOut[0]);
    System.out.println("Orientare dreapta : "+yOut[1]);
    System.out.println("Orientare jos    : "+yOut[2]);
    System.out.println("Orientare stanga : "+yOut[3]);
} //testare
} //class
```

4.2. Încărcarea șabloanelor dintr-un fișier text

Scriem un program care încarcă șabloanele de antrenare dintr-un fișier text, pentru o rețea neuronală ce are NI neuroni în stratul de intrare, NH neuroni în stratul ascuns și NO neuroni în stratul de ieșire. În final programul va afișa șabloanele încărcate în memorie.

Structura fișierului “sabloane.txt”:

```
1 1
1 1
1 1
1 0 0 0
0 1
1 1
0 1
0 1 0 0
1 1
1 0
1 1
0 0 1 0
1 1
0 1
1 1
0 0 0 1
```

```
//Reteaua NNI-NNH-NNO.
```

```
//Incarcare sabloane din fisier
```

```
import java.io.*;
```

```
import java.util.*;
class RN5
{
    static final int latimeS=2;//nr pixeli sablon pe latime
    static final int inaltimeS=3;
    static final int NS=4;//nr de sabloane
    static final int NI=latimeS*inaltimeS;//nr. neuroni intrare
    static final int NO=4;//nr neuroni iesire

    public static void main(String args[])
    {
        int sabloaneIn[][]=new int[NS][NI];
        /*
            sabloaneIn[i] este un vector ce are 2*3=6 valori,
            cati NI sunt, dar in fisier apare ca o matrice 2*3.
            Ex:
                1 1
                0 1
                0 0
            */
        int sabloaneOut[][]=new int[NS][NO];
        incarcaSabloane(sabloaneIn,sabloaneOut) ;
        afisareSabloane(sabloaneIn,sabloaneOut);
    }
    private static void incarcaSabloane(int sabloaneIn[], int
    sabloaneOut[])
    {
```

```
String numeF="sabloane.txt";
/* Structura fisier de sabloane, exemplu:
1 1
0 1
0 0
1 0 0 0
1 1
1 0
0 0
1 0 0 0
....
*/
//Presupunem corect scris fisierul de sabloane:
FileReader fr=null;
BufferedReader bfr=null;
try{
    fr=new FileReader(numeF);
    bfr=new BufferedReader(fr);
    //incarc cele NS sabloane:
    for(int i=0;i<NS;i++){
        //pt. sablonul i, mai intai citesc intrarile
        //lui:
        int j=0;//j este pentru fiecare neuron de
        //intrare
        for(int k=0;k<inaltimeS;k++){
            //un sablon are pe partea de
            //intrare, inaltimeS linii.
```

```
        String s=bfr.readLine();
        StringTokenizer tk=new
StringTokenizer(s);
        //sunt latimeS atomi in sirul s.
        for(int l=0;l<latimeS;l++){
        sabloaneIn[i][j]=Integer.parseInt(
        tk.nextToken());
        j++;
        }// for l
    }//for k
    //pt. sablonul i, acum citesc iesirile lui:
    //este o singura linie in fisier. Are NO
    //atomi(componente).
    String s=bfr.readLine();
    StringTokenizer tk=new StringTokenizer(s);
    for(int k=0;k<NO;k++)
        sabloaneOut[i][k]=Integer.parseInt(tk.nextT
        oken());
    }//for i
    bfr.close();
    fr.close();
}catch(IOException e){
    System.out.println(e);
    System.exit(1);
}
}
//incarcaSabloane()
```

```
private static void afisareSabloane(int sabloaneIn[][], int
sabloaneOut[][])
{

    for(int i=0;i<NS;i++){

        //afisare intrari sablon i:
        for(int k=0;k<inaltimeS;k++){
            for(int l=0;l<latimeS;l++)

                System.out.print(sabloaneIn[i][k*latimeS+l]+" ");
                System.out.println() ;
            }//for k

        //afisare iesiri sablon i:
        for(int k=0;k<NO;k++)
            System.out.print(sabloaneOut[i][k]);
        System.out.println();
    }//for i
} //afisareSabloane()
}
```

4.3. Clasificarea unor linii, după formă

Această aplicație ilustrează capacitatea unei rețele neuronale MLP cu un singur strat intern, de a clasifica intrări de tip analogic .

Se prezintă ca date de intrare pentru rețeaua neuronală, 10 valori numerice, numere reale pozitive. Ele reprezintă valorile ordonatelor unor puncte (pe axa Oy). Abscisele acestor puncte sunt egal distanțate. Cele 10 valori, reprezentate grafic, determină o curbă. Rețeaua neuronală trebuie să clasifice curba de pe intrările ei, într-una din următoarele clase:

- linie dreaptă de pantă zero (clasa 0)
- linie dreaptă de pantă pozitivă (clasa 1)
- linie dreaptă de pantă negativă (clasa 2)
- curbă cu concavitate pozitivă (clasa 3)
- curbă cu concavitate negativă (clasa 4)

În vederea rezolvării problemei propuse, am folosit o rețea MLP, total conectată, cu 10 neuroni în stratul de intrare, un singur strat ascuns cu 5 neuroni, și cu 5 neuroni în stratul de ieșire. Cele 5 ieșiri (Y_0, Y_1, Y_2, Y_3, Y_4) sunt asigurate fiecare, uneia din cele 5 clase de curbe:

Rețeaua a fost antrenată cu 15 șabloane (câte 3 pentru fiecare clasă de ieșire). Aceste șabloane sunt memorate în fișierul text `sabloane.txt` . Fiecare linie din acest fișier text conține 10 numere reale (valorile de pe intrările rețelei neuronale) și cele 5 valori corespunzătoare de pe neuronii de ieșire (0 sau 1). S-au folosit următoarele șabloane pentru antrenare:

2	2	2	2	2	2	2	2	2	2
1	0	0	0	0					
5	5	5	5	5	5	5	5	5	5
1	0	0	0	0					
7	7	7	7	7	7	7	7	7	7
1	0	0	0	0					
1	2	3	4	5	6	7	8	9	10
0	1	0	0	0					
2.5	3	3.5	4	4.5	5	5.5	6	6.5	7
0	1	0	0	0					
0.5	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
0	1	0	0	0					
10	9	8	7	6	5	4	3	2	1
0	0	1	0	0					
7	6.5	6	5.5	5	4.5	4	3.5	3	2.5
0	0	1	0	0					
9.5	8.5	7.5	6.5	5.5	4.5	3.5	2.5	1.5	0.5
0	0	1	0	0					
10	8	6	4	2	2	4	6	8	10
0	0	0	1	0					
9.5	7.5	5.5	3.5	1.5	1.5	1.5	4	6	8
0	0	0	1	0					
9	7	5	3	2	2.5	4	5	7	8
0	0	0	1	0					

2	4	6	8	9	9	8	6	4	2
0	0	0	0	1					
1	3	5	7	8	9	8.5	8	6	4
0	0	0	0	1					
3	3.5	5	7	7.5	8	7	6	5	2
0	0	0	0	1					

Fisierul de sabloane, sabloane.txt se editeaza cu editorul de texte Notepad..

Rețeaua a fost antrenată cu algoritmul standard de backpropagation.

Pentru valori numerice mari pe intrările rețelei , datorită funcției exponențiale din expresia funcției de transfer a unui neuron, se pot obține ca rezultate intermediare numere foarte mari, și deci erori de depășire (overflow) la execuția programului . De aceea valorile numerice de intrare (fie cele din șabloanele de antrenare, fie cele folosite la testarea rețelei antrenate) se preprocesează prin scalare. Se aduc în gama 0..1. Astfel , intrările neuronilor din primul strat sunt numere reale cuprinse în intervalul [0,1]. Formula de scalare folosită, este:

$$x_{Scalat} = (x - \min) / (\max - \min)$$
, unde max si min sunt, respectiv, cea mai mare și cea mai mică din cele 10 valori numerice.

Programul de simulare, permite salvarea ponderilor într-un fișier text , pe disc. De asemenea, testarea poate fi făcută folosind ponderile din memorie, rezultate în urma antrenării, sau folosind ponderile citite din fișierul text de ponderi.

Pentru eroarea globală de 0.01 rețeaua converge în aproximativ 1000 de epoci (evident, la fiecare rulare se obține alt rezultat). Pentru o eroare de 0.001, șabloanele de antrenare sunt învățate în aproximativ 8000 de epoci.

```
import java.util.*;
import java.io.*;
public class RNLinii {

    static final int NR_NEURONI_IN=10;
    static final int NR_NEURONI_OUT=5;
    static final int NR_NEURONI_HIDDEN=10;
    static final int NR_TOTAL_SABLOANE=3*NR_NEURONI_OUT;
    static final double EROARE_IMPUSA=0.01;
    static final int NR_MAXIM_EPOCI=20000;
    static double sabloaneIn[][]=
        new double[NR_TOTAL_SABLOANE][NR_NEURONI_IN];
    static double sabloaneOut[][]=
        new double[NR_TOTAL_SABLOANE][NR_NEURONI_OUT];
    static double w1[][]=
        new double[NR_NEURONI_HIDDEN][NR_NEURONI_IN];
    static double w2[][]=
        new double[NR_NEURONI_OUT][NR_NEURONI_HIDDEN];
    public static void main(String args[])
    {
        boolean converge;
        ConsoleReader cr=new ConsoleReader(System.in);
```

```
char c,raspuns;
do{
    System.out.print("a (antrenare) sau t (testare) ? [a/t] : ");
    c=cr.readChar();
}while((c!='a')&&(c!='t'));
switch(c){
    case 'a'://antrenare
        initPonderi();
        incarcaSabloane();
        converge=antrenare();
    if(!converge)
        System.out.println(
            "Nu a converș in nr. de epocii specificat !");
    else{
        System.out.println("A converș. !");
        afisareExtremePonderi();
        System.out.print("Salvare ponderi in fisier ? [d/n] : ");
        raspuns=cr.readChar();
        if(raspuns=='d')salvare();
        for(;;){
            System.out.print("Testare ? [d/n] : ");
            raspuns=cr.readChar();
            if(raspuns=='d')testare();
            else break;
        }//for
    }//else a converș
    break; //antrenare
```

```
        case 't': //testare
            incarcaPonderi();
            for(;;){
                testare();
                System.out.print("Testare ? [d/n] : ");
                raspuns=cr.readChar();
                if(raspuns=='n')break;
            }//for
            break;//testare
    }//switch
}

private static void initPonderi()
{
    int i,j;
    Random r=new Random();
    //initializare primul strat de ponderi:
    for(i=0;i<NR_NEURONI_HIDDEN;i++)
        for(j=0;j<NR_NEURONI_IN;j++)
            w1[i][j]=r.nextDouble()-0.5;
    //initializare strat al doilea de ponderi:
    for(i=0;i<NR_NEURONI_OUT;i++)
        for(j=0;j<NR_NEURONI_HIDDEN;j++)
            w2[i][j]=r.nextDouble()-0.5;
}

private static void incarcaSabloane()
{

```

```
try{
    FileReader fp=new FileReader("sabloane.txt");
    BufferedReader fpBuf=new BufferedReader(fp);
    //Stim ca fisierul contine 2 * NR_TOTAL_SABLOANE de linii.
    //(pentru fiecare sablon, sunt 2 linii: linia cu intrarile sablonului
    // si linia cu iesirile sablonului.)
    for(int ex=0;ex<NR_TOTAL_SABLOANE;ex++){
        //pt. fiecare sablon ex, citim 2 linii:
        String linieCrt=fpBuf.readLine();
        //S-a citit linia cu intrarile sablonului ex.
        if(linieCrt==null){
            System.out.println(
                "Structura fisierului de sabloane nu este corecta!");
            System.exit(1);}
        //Citim valorile din sablon, pt. fiecare neuron de intrare:
        /*
        Pentru a separa un string in articolele sale componente
        (separator implicit este caracterul blank), se foloseste clasa
        StringTokenizer si metoda nextToken( )
        .*/
        StringTokenizer st=new StringTokenizer(linieCrt);
        for(int i=0;i<NR_NEURONI_IN;i++){

            double x=Double.parseDouble(st.nextToken());
            //Cea mai mare valoare din fisierul de sabloane: 10.0
            //Cea mai mica valoare din fisierul de sabloane: 0.0
            x=(x-0.0)/(10.0-0.0);//scalare in plaja 0.0, 1.0
```

```
sabloaneIn[ex][i]=x;}
//Citim linia cu iesirile sablonului ex:
linieCrt=fpBuf.readLine();
if(linieCrt==null){
    System.out.println(
        "Structura fisierului de sabloane nu este corecta!");
    System.exit(1);}
//Citim valorile din sablon, pt. fiecare neuron de iesire:
st=new StringTokenizer(linieCrt);
for(int i=0;i<NR_NEURONI_OUT;i++)
    sabloaneOut[ex][i]=
        Double.parseDouble(st.nextToken())
}
}

private static boolean antrenare()
{
    double yHidden[]=new double[NR_NEURONI_HIDDEN];
    double yOut[]=new double[NR_NEURONI_OUT];
    double eroareIesireSablonCrt[]=
        new double[NR_TOTAL_SABLOANE];
    double eroareTotala;
    int nrEpoci=0;
```

```
boolean converge=false;
boolean stop=false;
int i,ex;
while(!stop){
    for(ex=0;ex<NR_TOTAL_SABLOANE;ex++){
        calculIesiriNeuroni(yHidden,yOut,ex);
        eroareIesireSablonCrt[ex]=0;
        for(i=0;i<NR_NEURONI_OUT;i++){
            eroareIesireSablonCrt[ex]+=0.5*(yOut[i]-
                sabloaneOut[ex][i])*(yOut[i]-sabloaneOut[ex][i]);
            modificaPonderi(yOut,yHid
        }
    }
    nrEpoci++;
    eroareTotala=0;
    for(ex=0;ex<NR_TOTAL_SABLOANE;ex++){
        eroareTotala+=eroareIesireSablonCrt[ex];
    }
    System.out.println("Epoca nr: "+nrEpoci+"
        eroare totala= "+eroareTotala);
    if(eroareTotala <= EROARE_IMPUSA){
        converge=true;
        stop=true;}
    if(nrEpoci > NR_MAXIM_EPOCI)stop=true;
}

return converge;
}

private static void calculIesiriNeuroni(double yHidden[],
```

```

        double yOut[],int ex)
    {
        int i,j;
        double xTotal_i;
        for(i=0;i<NR_NEURONI_HIDDEN;i++){
            xTotal_i=0;
            for(j=0;j<NR_NEURONI_IN;j++){
                xTotal_i=xTotal_i+sabloaneIn[ex][j]*w1[i][j];
                yHidden[i]=1/(1+Math.exp(-xTotal_i));
            }//for i
            for(i=0;i<NR_NEURONI_OUT;i++){
                xTotal_i=0;
                for(j=0;j<NR_NEURONI_HIDDEN;j++){
                    xTotal_i=xTotal_i+yHidden[j]*w2[i][j];
                    yOut[i]=1/(1+Math.exp(-xTotal_i));
                }//for i
            }
        }

        private static void modificaPonderi(double yOut[],
            double yHidden[],int ex)
        {
            double deltaOut[]=new double[NR_NEURONI_OUT];
            double deltaHidden[]=new double[NR_NEURONI_HIDDEN];
            int i,j,k;
            double sumaPonderataDeltaFii;
            double alfa=1.0;
            double eta=1.0;

```

```
for(i=0;i<NR_NEURONI_OUT;i++)
    deltaOut[i]=(yOut[i]-sabioaneOut[ex][i])*(1-yOut[i])*yOut[i];
for(i=0;i<NR_NEURONI_OUT;i++)
    for(j=0;j<NR_NEURONI_HIDDEN;j++)
        w2[i][j]=alfa*w2[i][j]-eta*deltaOut[i]*yHidden[j];
for(i=0;i<NR_NEURONI_HIDDEN;i++){
    sumaPonderataDeltaFii=0;
    for(j=0;j<NR_NEURONI_OUT;j++)
        sumaPonderataDeltaFii+=w2[j][i]*deltaOut[j];
    deltaHidden[i]=sumaPonderataDeltaFii*
        (1-yHidden[i])*yHidden[i];
}
for(i=0;i<NR_NEURONI_HIDDEN;i++)
    for(j=0;j<NR_NEURONI_IN;j++)
        w1[i][j]=alfa*w1[i][j]-
            eta*deltaHidden[i]*sabioaneIn[ex][j];
} //modificaPonderi

private static void afisareExtremePonderi()
{
    int i,j;
    double wMax,wMin;
    wMax=w1[0][0];
    wMin=w1[0][0];
    for(i=0;i<NR_NEURONI_HIDDEN;i++)
        for(j=0;j<NR_NEURONI_IN;j++)
        {
```

```

        if(w1[i][j] > wMax)wMax=w1[i][j];
        if(w1[i][j] < wMin)wMin=w1[i][j];
    }
    for(i=0;i<NR_NEURONI_OUT;i++)
        for(j=0;j<NR_NEURONI_HIDDEN;j++)
        {
            if(w2[i][j] > wMax)wMax=w2[i][j];
            if(w2[i][j] < wMin)wMin=w2[i][j];
        }
    System.out.println("Ponderea maxima="+wMax+
        " ponderea minima="+wMin);
}

private static void testare()
{
    int i;
    //iesiri neuroni din strat hidden:
    double yHidden[]=new double[NR_NEURONI_HIDDEN];
    //iesiri neuroni din strat iesire (cele calculate):
    double yOut[]=new double[NR_NEURONI_OUT];
    double valoriCurba[]=new double[10];
    double max,min;
    System.out.println("Dati cele 10 valori pozitive ale curbei:");
    //In timp ce le citim, aflam si minimul si maximul dintre ele
    // pentru a face scalarea:
    ConsoleReader cr=new ConsoleReader(System.in);
    System.out.print("Valoarea curbei in punctul 0 : ");

```

```
valoriCurba[0]=cr.readDouble();
max=valoriCurba[0];
min=valoriCurba[0];
//Celelalte 9 valori:
for(i=1;i<10;i++){
    System.out.print("Valoarea curbei in punctul "+i+" : ");
    valoriCurba[i]=cr.readDouble();
    if(valoriCurba[i] > max)max=valoriCurba[i];
    if(valoriCurba[i] < min)min=valoriCurba[i];
}
//Metoda calculIesiriNeuroni(), calculeaza iesirile din retea,
//avand ca date la intrari, datele din sabloaneIn[][]
//De aceea, incarcam datele citite in sabloaneIn[0][]:
if(max!=min)//pt. a nu avea impartire cu 0:
    for(i=0;i<10;i++){
        sabloaneIn[0][i]=(valoriCurba[i]-min)/(max-min);
    }
else for(i=0;i<10;i++){
    sabloaneIn[0][i]=1;
}
calculIesiriNeuroni(yHidden,yOut,0);
System.out.println("Dreapta, panta 0 : "+yOut[0]);
System.out.println("Dreapta, panta poz. : "+yOut[1]);
System.out.println("Dreapta, panta neg. : "+yOut[2]);
System.out.println("Dreapta, concav. poz. : "+yOut[3]);
System.out.println("Dreapta, concav. neg. : "+yOut[4]);
}

private static void salvare()
```

```
{  
    StringBuffer nume=new StringBuffer("ponderi.");  
    System.out.print("Introduceti extensia numelui fisierului: ");  
    ConsoleReader cr=new ConsoleReader(System.in);  
    String extensie=cr.readLine();  
    nume.append(extensie);  
    String numeF=nume.toString();  
    int i,j;  
    try{  
        FileWriter fp=new FileWriter(numeF);  
        BufferedWriter fpBuf=new BufferedWriter(fp);  
        for(i=0;i<NR_NEURONI_HIDDEN;i++){  
            for(j=0;j<NR_NEURONI_IN;j++){  
                String s=w1[i][j]+" ";  
                fpBuf.write(s,0,s.length());}  
            fpBuf.newLine();  
        }  
        for(i=0;i<NR_NEURONI_OUT;i++){  
            for(j=0;j<NR_NEURONI_HIDDEN;j++){  
                String s=w2[i][j]+" ";  
                fpBuf.write(s,0,s.length());}  
            fpBuf.newLine();  
        }  
        fpBuf.close();  
    }catch(IOException e){  
        System.out.println(e);  
        System.exit(1);}  
}
```

```
}
```

```
private static void incarcaPonderi()
{
    StringBuffer nume=new StringBuffer("ponderi.");
    System.out.println(
        "Numele fisierului din care citim ponderile este:ponderi.xxx ");
    System.out.print("Introduceti extensia numelui fisierului: ");
    ConsoleReader cr=new ConsoleReader(System.in);
    String extensie=cr.readLine();
    nume.append(extensie);
    String numeF=nume.toString();
    int i,j;
    try{
        FileReader fp=new FileReader(numeF);
        BufferedReader fpBuf=new BufferedReader(fp);

        //Citim ponderile w1:
        for(i=0;i<NR_NEURONI_HIDDEN;i++){
            //Toate ponderile neuronului hidden i, sunt scrise pe o linie text.
            //Toate ponderile neuronului hidden i+1 sunt scrise pe linia
            // urmatoare.
            String linieCrt=fpBuf.readLine();
            if(linieCrt==null){
                System.out.println(
                    "Structura fisierului de ponderi nu este corecta!");
                System.exit(1);}
        }
```

```

StringTokenizer st=new StringTokenizer(linieCrt);
for(j=0;j<NR_NEURONI_IN;j++)
w1[i][j]=Double.valueOf(st.nextToken()).doubleValue();}for i
//Citim ponderile w2:
for(i=0;i<NR_NEURONI_OUT;i++){
String linieCrt=fpBuf.readLine();
if(linieCrt==null){
System.out.println("
    Structura fisierului de ponder nu este corecta!");
System.exit(1);}
StringTokenizer st=new StringTokenizer(linieCrt);
for(j=0;j<NR_NEURONI_HIDDEN;j++)
w2[i][j]=Double.valueOf(st.nextToken()).doubleValue();
}for i
fpBuf.close();
}catch(IOException e){
System.out.println(e);
System.exit(1);}
}end incarcaPonderi
}class

```

Pentru a citi date de la tastatura, s-a dezvoltat clasa ConsoleReader.

```

import java.io.*;

public class ConsoleReader{
    //obiect pentru a citi o linie de caractere

```

```
private BufferedReader buff_reader;

//constructorul clasei
public ConsoleReader(InputStream inSm)
{
    //creeaza obiectul buff_reader ce are capacitatea de a citi o linie
    buff_reader=new BufferedReader(
        new InputStreamReader(inSm));
}

//metodele clasei
public String readLine()
{
    String inStr="";
    try{
        inStr=buff_reader.readLine();

        }catch(IOException e){
            System.out.println(e);
            System.exit(1);

        }

    return inStr;
}

public int readInt()
{
    String inStr=this.readLine();
    int nr=Integer.parseInt(inStr);
```

```
    return nr;

}

public char readChar()
{
    String inStr=this.readLine();
    return inStr.charAt(0);
}

public double readDouble()
{
    String inStr=this.readLine();
    double x=Double.parseDouble(inStr);
    return x;
}
}
```

Câteva exemple cu rezultatele experimentale obținute la testare: (s-au folosit ponderile memorate în fișierul liniw.001, rezultate în urma antrenării cu eroare globală de 0.001)

- Pentru intrările: 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 4.5 se obține la ieșire, la neuronul corespunzător clasei “dreaptă de pantă zero”, valoarea 0.999159 , deci foarte apropiată de valoarea ideală 1.0. La ceilalți neuroni de ieșire , se obțin valori foarte mici, practic egale cu 0.

- Pentru intrările: 1.8 1.8 2.0 2.0 1.8 1.8 1.8 2.0 2.0 1.7 se obține la ieșire, la neuronul corespunzător clasei “dreaptă de pantă zero”, valoarea 0.998988 , deci foarte apropiată de valoarea ideală 1.0.

La ceilalți neuroni de ieșire , se obțin valori foarte mici, practic egale cu 0.

- Pentru intrările: 11 22 33 44 55 66 77 88 99 110 se obține la ieșire, la neuronul corespunzător clasei “dreaptă de pantă pozitivă”, valoarea 0.998665 , deci foarte apropiată de valoarea ideală 1.0. La ceilalți neuroni de ieșire , se obțin valori foarte mici, practic egale cu 0.

- Pentru intrările: 11 22 33 44 44 44 25 25 20 10 se obține la ieșire, pe neuronul corespunzător clasei “curbă cu concavitate negativă”, valoarea 0.986690 , deci foarte apropiată de valoarea ideală 1.0. La ceilalți neuroni de ieșire , se obțin valori foarte mici, practic egale cu 0.

- Pentru intrările: 100 80 75 70 30 30 35 50 55 90 se obține la ieșire, la neuronul corespunzător clasei “curbă cu concavitate pozitivă”, valoarea 0.955646 , deci foarte apropiată de valoarea ideală 1.0. La ceilalți neuroni de ieșire , se obțin valori foarte mici, practic egale cu 0.

- Pentru intrările: 100 95 73 70 65 60 50 45 40 35 se obține la ieșire, la neuronul corespunzător clasei “dreaptă de pantă negativă”, valoarea 0.995146 , deci foarte apropiată de valoarea ideală 1.0. La ceilalți neuroni de ieșire , se obțin valori foarte mici, practic egale cu 0.

4.4. Aproximare a funcțiilor neliniare

Pentru a ilustra capacitatea rețelelor neuronale de a aproxima funcții neliniare, alegem ca exemplu de lucru aproximarea funcției $f(x) = x^2 : (0,1) \rightarrow (0,1)$. Vom folosi o rețea neuronală cu un singur neuron în stratul de intrare, două straturi ascunse, având NH1 și NH2 neuroni și un singur neuron în stratul de ieșire.

Rețeaua va fi antrenată pentru a învăța 100 de șabloane.

//Reteaua 1-NH1-NH2-1

//aproximare functie $f(x)=x^2 : (0,1) \rightarrow (0,1)$

class RN4

{

public static void main(String args[])

{

final int NH1=7;//nr. neuroni din primul strat ascuns

final int NH2=7;//nr. neuroni din al doilea strat ascuns

final int NS=100;//nr. total de șabloane

final double E_TOTAL=0.01;//eroarea de învățare a

//șabloanelor

//(experimental: 1700 de epoci)

double xS[]=new double[NS];//vectorul de intrari ale

//șabloanelor

double yS[]=new double[NS];//vectorul de iesiri ale

//șabloanelor

initȘabloane(xS,yS);

int i,j,k;

//primul strat de ponderi:

```
double w1[]=new double[NH1];
for(i=0;i<NH1;i++)
    w1[i]=Math.random()-0.5;
//al doilea strat de ponderi:
double w2[][]=new double[NH2][NH1];
for(i=0;i<NH2;i++)
    for(j=0;j<NH1;j++)
        w2[i][j]=Math.random()-0.5;
//al treilea strat de ponderi:
double w3[]=new double[NH2];
for(i=0;i<NH2;i++)
    w3[i]=Math.random()-0.5;
//iesirile din primul strat de neuroni ascunsi:
double yH1[]=new double[NH1];
//iesirile din al doilea strat de neuroni ascunsi:
double yH2[]=new double[NH2];
//erorile de invatare pentru fiecare sablon:
double e[]=new double[NS];
int nrEpoci=0;
for(;;){
    for(i=0;i<NS;i++){
        //propagare inainte sablon i:
        //calcul yH1[]:
        for(j=0;j<NH1;j++){
            double a=xS[i]*w1[j]; //activarea in neuronul j
            yH1[j]=1/(1+Math.exp(-a));
        }
    }
}
```

```
//calcul yH2[]:
for(j=0;j<NH2;j++){
    double a;//activarea in neuronul j
    a=0;
    for(k=0;k<NH1;k++){
        a=a+w2[j][k]*yH1[k];
        yH2[j]=1/(1+Math.exp(-a));
    }
//calculul iesirii in neuronul de iesire:
double y;
double a;//activarea in neuronul de iesire
a=0;
for(j=0;j<NH2;j++){
    a=a+w3[j]*yH2[j];
    y=1/(1+Math.exp(-a));
//S-a terminat etapa de propagare inainte.
//Calcul eroare de inavatare a sablonului
//curent:
e[i]=0.5*(y-yS[i])*(y-yS[i]);
//Propagare inapoi:
double delta;//in neuronul de iesire
delta=(y-yS[i])*y*(1-y);
//actualizare strat 3 de ponderi:
for(j=0;j<NH2;j++){
    w3[j]=w3[j]-yH2[j]*delta;
//valorile delta in neuronii din al doilea strat
//ascuns:
```

```
double deltaH2[]=new double[NH2];
for(j=0;j<NH2;j++)
    //neuronul j are un singur fiu
    deltaH2[j]=w3[j]*delta*yH2[j]*(1-yH2[j]);
//actualizare strat doi de ponderi:
for(j=0;j<NH2;j++)
    for(k=0;k<NH1;k++)
        w2[j][k]=w2[j][k]-yH1[k]*deltaH2[j];
//valorile delta in neuronii din primul strat
//ascuns:
double deltaH1[]=new double[NH1];
for(j=0;j<NH1;j++){
    //neuronul j are NH2 fii
    double sumaPonderataDeltaFii=0;
    for(k=0;k<NH2;k++)
        sumaPonderataDeltaFii=
        sumaPonderataDeltaFii+w2[k][j]*deltaH2[k];
    deltaH1[j]=
    sumaPonderataDeltaFii*yH1[j]*(1-yH1[j]);
}
//actualizare strat unu de ponderi:
for(j=0;j<NH1;j++)
    w1[j]=w1[j]-xS[i]*deltaH1[j];
}for i;
nrEpoci++;
double E;//eroare de invatare set de sabloane
E=0;
```

```
        for(i=0;i<NS;i++)
            E=E+e[i];
        System.out.println("epoca: "+nrEpoci+" eroare de
        invatare="+E);
        if(E<=E_TOTAL)break;
    }//for;;

} //main
private static void initSabloane(double xS[],double yS[])
{
    int dim=xS.length;
    for(int i=0;i<dim;i++){
        xS[i]=i/(double)dim;
        yS[i]=xS[i]*xS[i];
    }
}
}
```
